Generality Is Compression: An Operational Measure, and a Representation That Fails It

Shehzad Ahmed
Independent University, Bangladesh

September 2026

Abstract

"General" is used to mean two different things. One is *breadth*: the system handles many cases because many cases were stored. The other is *generality*: the system handles many cases because one structure covers them. The distinction is standard as intuition—the novice sorts physics problems by surface features, the expert by governing principle—and is usually left there.

We give it an operational measure. If generality is compression, then a general representation should hold description length roughly fixed while the world it describes grows, and its *coverage per stored item* should therefore rise with environment size, while a merely broad representation's falls. We test this on an agent whose memory can be indexed two ways that differ in nothing but the key: by *location*, or by *local situation*. Across environments of 25, 64, and 144 states, coverage per stored item moves in opposite directions, $0.99 \to 0.83 \to 0.75$ for location and $2.13 \to 4.12 \to 6.93$ for situation. The situation-keyed store also crosses *below* the location-keyed one in size while covering seven times as many cases per item.

The negative half is sharper than the positive. Location-keyed memory does not merely fail to transfer to unseen environments—it *harms*, at every scale, and is worse than a store whose contents have been randomly shuffled. The asymmetry has a mechanism: a shuffled store is harmless because it rarely matches and therefore rarely fires, whereas the location-keyed store fires constantly and the world behind its matches has moved. **It retrieves reliably and retrieves wrong.** All seven predictions were preregistered and held on 20,000 seeds disjoint from any exploratory range.

1 Two things called general

A chess engine plays chess. A translator translates. Adding both to a system makes it handle more cases, and nothing about the second helps with the first. Call this **breadth**: coverage purchased by accumulation.

Contrast the classic finding on expertise. Given a set of physics problems, novices sort them by surface—inclined planes here, pulleys there—while experts sort the same problems by governing principle, placing a ramp and a pulley together because both are conservation of energy. The expert's organisation is smaller and covers more. Call this **generality**: coverage purchased by structure.

The distinction is intuitive and rarely measured. This paper offers a measure and applies it to a case where the two representations differ in exactly one respect, so that the comparison is clean.

2 Why compression is the right test

The reason to expect compression to separate them is not analogy. It follows from what compression is.

Shannon's entropy $H(P) = \sum_i p_i \log(1/p_i)$ is the minimum average bits per symbol achievable if one's code matches the world. Coding a world P with a scheme optimised for Q costs the cross-entropy $H(P, Q) = \sum_i p_i \log(1/q_i)$, which exceeds $H(P)$ by the Kullback–Leibler divergence and is minimised exactly when $Q = P$.

That minimisation property is not one convenient choice among many. If one asks which loss F makes $\sum_i P_i F(Q_i)$ minimal at $Q = P$ subject to $\sum_i Q_i = 1$, the Lagrange condition gives $P_i F'(Q_i) = \lambda$, hence $F'(x) \propto 1/x$ and $F = -\log$ uniquely. Cross-entropy is forced.

Two consequences matter here. First, prediction and compression are the same operation: any predictor Q converts, via arithmetic coding, into a compressor achieving approximately $\sum -\log Q$ bits, so minimising predictive loss is minimising code length. Second, description length is the natural formalisation of "one structure under many things"—the Kolmogorov intuition that the shortest program producing a body of data is the structure in it.

Breadth and generality are therefore distinguishable in principle by how description length behaves as the described world grows. Breadth must grow with it: each new case is a new entry. Generality need not, because the entries were never about cases.

3 An operational measure

Practical description length is not computable, but a proxy is. For a system with an explicit store of N items that are retrieved during operation, define

$$\text{coverage} \;=\; \frac{\text{retrievals per episode}}{N}$$

the number of situations each stored item is actually used on. A representation whose items name individual cases has coverage near one and falling: as the world grows there are more cases, each item still names one, and the chance of revisiting any particular case declines. A representation whose items name recurring *situations* has coverage above one and rising: the number of distinct situations does not grow with the size of the world, so each item is matched more often.

This yields the prediction we test:

> **As environment size grows, coverage per stored item falls for a case-indexed representation and rises for a structure-indexed one.**

The prediction is about a *direction of change with scale*, which is harder to satisfy by accident than a difference at one size.

4 Experiment

4.1 Task and representations

An agent traverses a grid whose obstacles are re-randomised for every episode, so route memorisation cannot help. During training it stores experiences and consults the store when scoring candidate moves. We vary only the key:

- **Location key:** `at (x,y)` → `move`. Names the cell.
- **Situation key:** `blk<4 bits>|goal(dx,dy)` → `move`. Names which of the four neighbours are blocked, plus the coarse direction of the goal.

The agent already computes neighbour blockage in order to bias toward the goal, so the situation key uses *no information the location key lacked*. The two conditions are identical agents differing in one string.

4.2 Transfer design

Training runs on four novel environments. The agent then attempts one it has never seen, three times from the same trained state, differing only in the store: **intact**, **wiped** (empty), or **shuffled** (same size, same keys, values permuted). The shuffled arm is the control that separates a store's *content* from the mechanics of having a store at all.

Goal bias is held constant across arms and the held-out episode uses an identical random stream, so the three arms are paired and the statistic is the per-seed difference.

4.3 Preregistration

Seven predictions with numeric thresholds—set below the exploratory point estimates so the test is a replication rather than a restatement—were committed to version control before the first replication seed was drawn, along with the analysis plan and a stopping rule of one run. Seeds 500001–520000 are disjoint from every range used in exploratory work. The primary outcome was declared to be the intact-versus-*shuffled* contrast, not intact-versus-wiped: the question is whether content carries structure, not whether having a store does anything.

5 Results

Coverage diverges, as predicted. Location: 0.99 → 0.83 → 0.75. Situation: 2.13 → 4.12 → 6.93. The environment grew 5.8× in area; the location store grew 2.6× and the situation store 1.7×. Sub-linear, not flat—the honest statement—but growing far more slowly than the world it covers.

The stores cross over. At 25 states the situation store is *larger* (8.51 vs 6.77). At 144 states it is *smaller* (14.43 vs 17.24) while covering seven times as many situations per item. A representation that starts out costlier and ends up cheaper, while covering more, is the signature the measure was built to detect.

States	Key	Store	Retrievals	**Coverage**	intact − wiped	intact − shuffled
25	location	6.77	6.7	**0.99**	−0.0108	−0.0126
25	situation	8.51	18.1	**2.13**	+0.0425	+0.0309
64	location	11.55	9.6	**0.83**	−0.0040	−0.0042
64	situation	12.05	49.6	**4.12**	+0.0442	+0.0352
144	location	17.24	13.0	**0.75**	−0.0043	−0.0029
144	situation	14.43	100.0	**6.93**	+0.0207	+0.0159

Table 1: Held-out transfer, $n = 20{,}000$ paired seeds per cell. All six differences exclude zero. All seven preregistered predictions held.

Transfer follows coverage. Situation-keyed memory helps at every size against both controls, including the shuffled one. Location-keyed memory helps at no size.

6 The negative half

The sharper result is not that location keys fail to help. It is that they *harm*, and that they are worse than randomly shuffled contents.

> A shuffled store is harmless *because* it is wrong at random: its keys rarely match the current state, so it rarely fires. The location store fires constantly—it was learned from the same *form* of experience it is now consulted about—and the world behind those matches has changed.

The obstacles it learned to route around are elsewhere now, so its guidance is systematically, not randomly, wrong. It steers the agent around obstacles that are absent and into ones that are present.

Two things follow.

A memory's capacity for harm scales with its coverage. High-coverage memory is high-leverage in both directions. This is uncomfortable for the obvious reason: coverage is also the thing that makes memory worth having.

Retrieval frequency is not evidence of value. It is highest exactly where the representation is most misleading—in a shifted environment, which is the regime memory exists to serve. Reporting hit rate as a virtue inverts the relationship in the case that matters.

The generalisation we would defend is a design criterion rather than a law: *index on whatever is invariant in the domain.* Cell coordinates are not invariant when obstacles move; local geometry is. In a reflexive setting—one where acting changes the environment—what stays fixed is rarely the state and more often the relation, which suggests relational indexing is the default rather than the refinement.

7 Relation to the companion result

The same data appear in a companion paper as one of five demonstrations that a component's *activity* is not evidence of its *contribution*. The overlap is deliberate and we state it plainly: there, the location store is an instance of a mechanism that fires and does harm; here, the contrast between the two keys is used to make generality measurable. The experiment is one study; the two papers ask different questions of it.

8 Limits

One task family. A grid navigator with random-utility action selection. The measure is defined for any system with an explicit retrievable store, but it has been applied once.

Small effects. The transfer differences are a few percentage points of success rate. The intervals exclude zero because n is large, not because the effects are big. The *coverage* result is the striking one—a factor of seven at the largest size—and it is a property of the representation rather than of the outcome.

Sub-linear, not invariant. We predicted the situation store would stay roughly flat. It grew 1.7×. The prediction was directionally right and quantitatively wrong, and we record it as such.

The measure needs a store. Coverage per item is defined only for systems with explicit, countable, retrievable entries. It does not apply to a representation distributed across weights, which is the case one would most like to measure. Whether an analogue exists there is open, and we think it is the interesting question this work does not answer.

9 Conclusion

Breadth is coverage bought by accumulation; generality is coverage bought by structure. If generality is compression, the difference should appear as description length holding steady while the world grows—and it does, as coverage per stored item moving in opposite directions with scale.

The result also carries a warning that the positive framing hides. The failing representation was not inert. It retrieved on nearly every step, and its retrieving is precisely what made it harmful. Between a representation that covers many cases and one that covers many *situations* lies the whole distance between a store that transfers and a store that misleads, and from the inside—at training time, in the environment where it was learned—the two are indistinguishable.

Reproducibility. The preregistration, lock, runner, and result file are in the accompanying repository. The lock hashes both the preregistration and the analysis code, and the runner verifies those hashes at startup and aborts if either has changed.